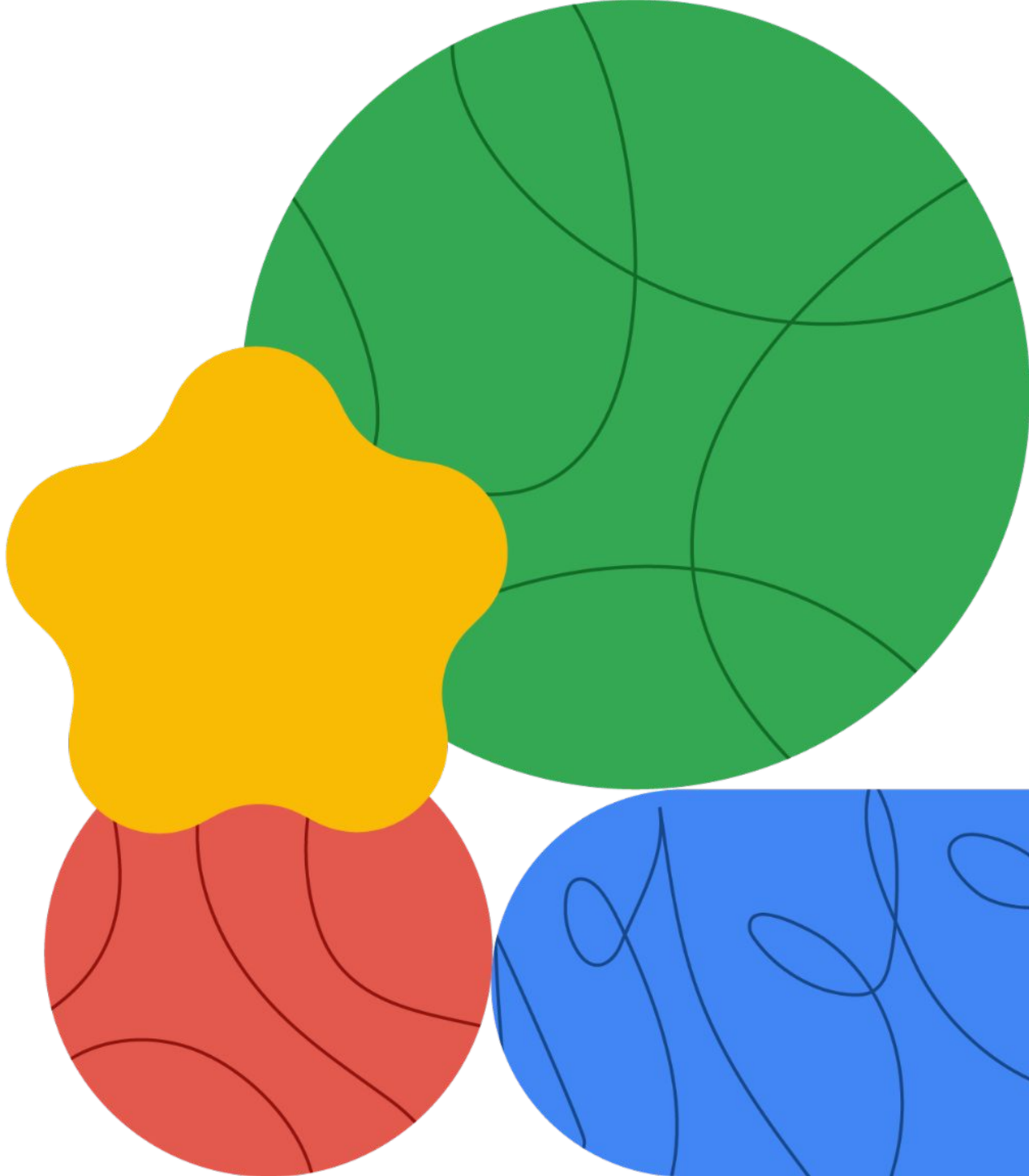
Google Cloud

Agentic AI with the Google Agent Development Kit (ADK)

Skills Validation Workshop



Proprietary + Confidential

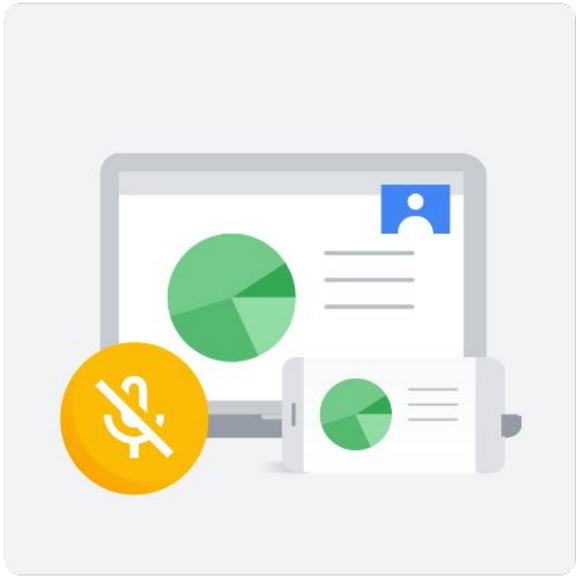
Your Instructor

- Background
- Position
- Organization



Proprietary + Confidential

Etiquette



Mute microphone



No recording



Ask questions

Proprietary + Confidential

Course format



Instructor Lectures
(10%)



Challenge Labs
(90%)

Proprietary + Confidential

Why should I participate in this Skills Validation Workshop?

01 Validate the knowledge you gained from the Google Agent Development Kit training program

02 Prove your ability to implement generative AI agents for customers

03 Demonstrate a broad understanding of using Google Cloud tools to implement real-world generative AI agents

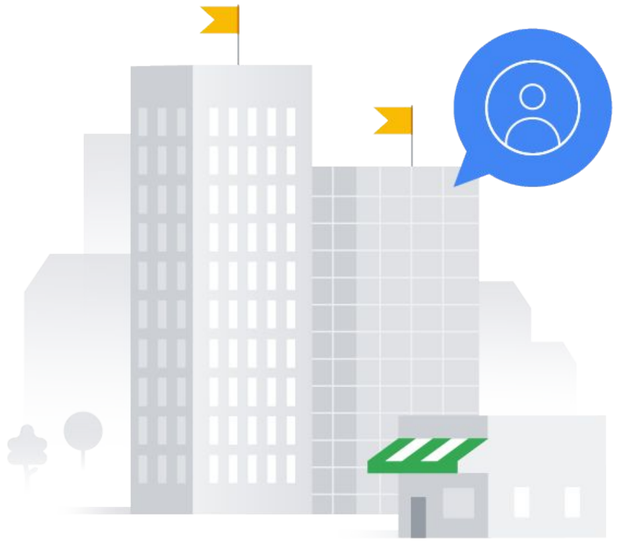
04 Get more work for you and your company!



Proprietary + Confidential

Target audience

This workshop assumes intermediate to advanced knowledge of generative AI, Google Cloud, Google Gemini, and related APIs



Google Cloud

- ML Engineers
- Developers
- Solution Architects

Proprietary + Confidential

Prerequisites

- Generative AI Foundational training in Google Cloud Skills Boost
- Experience deploying solutions to Google Cloud
- Working knowledge of Google Agent Development Kit, Agent Platform, Gemini, and related APIs
- Programming experience with Python and Jupyter Notebooks



Proprietary + Confidential



Agenda



Day 1

9:00	Introduction
9:30	Challenge 1: Real-Time Weather Alerts Agent
11:00	Challenge 2: Enhancing Agents with Callbacks
1:00	Challenge 3: Developing Multi-Agent Systems
3:00	Challenge 4: Programming an Agent Workflow

Day 2

9:00	Bonus Challenge 5: Deploying Agents
11:00	Challenge 6: Emergency Preparedness Assistant

Proprietary + Confidential

Grading

Challenges	Points
Challenge One: Real-Time Weather Alerts Agent	15
Challenge Two: Enhancing Agents with Callbacks	15
Challenge Three: Developing Multi-Agent Systems	15
Challenge Four: Programming an Agent Workflow	15
Bonus Challenge Five: Deploying Agents	10
Challenge 6: Emergency Preparedness Assistant	40
Total points possible:	**110**

- 80 points is required to pass the workshop

Proprietary + Confidential

Submitting your work...

01 Log into **GitHub** (if you do not have an account, create one)

02 Create a public GitHub repository named:
agent-dev-skills-workshop-<<your-name>>

03 Challenges will be done using Jupyter Notebooks or Python programs. You will need to upload your code to this repository for grading.

04 Make this repo available to your instructor.



Proprietary + Confidential

Open Google Cloud Skills Boost

01 Open your preferred browser, preferably Chrome

02 Go to the Skills Boost URL https://explore.qwiklabs.com/discovery

03 Sign in with existing account, or join with new account, using the **email you used to register** for the course.

04 Launch the course from **In Progress** section of your home page.

Access issues

If you encounter any problems getting into Skills Boost, please let your instructor know.

Proprietary + Confidential



Challenge One

Real-Time Weather Alerts Agent

Proprietary + Confidential

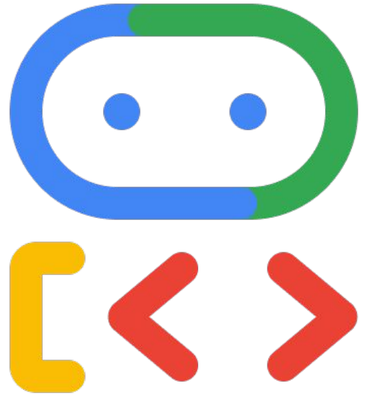
What is Agentic AI?

- Agentic AI refers to systems that use intelligent agents to autonomously perceive, reason, and act in a coordinated way to achieve complex goals
- Each agent has a specific role and can use tools, memory, and planning to complete tasks
- Agents can be combined into multi-agent systems to handle complex workflows
  - Coordination strategies include sequential, parallel, and looping agents
- Agentic systems are dynamic, goal-driven, and capable of decision-making based on context

Proprietary + Confidential

Using the Google Agent Development Kit (ADK)

- The Google Agent Development Kit (ADK) enables developers to build, test, and deploy intelligent agent-based applications using Gemini models.
- Provides a framework to define agents, tools, and workflows with Python
- Supports integration with Google Cloud services and Google Agent Platform for scalable deployment
- Includes runners and lifecycle hooks to manage agent behavior and execution flow



Proprietary + Confidential

Creating agents with Gemini models

```
from google.adk.agents import Agent

code_agent = Agent(
    name="Joey",
    model="gemini-#.#-flash",
    description=(
        "You answer programming questions."
    ),
    instruction=(
        """You answer questions related to programming.
        Answer coding questions using Python unless told otherwise."""
    )
)
```

Proprietary + Confidential

Using models from other vendors

```
from google.adk.agents import Agent
from google.adk.models.lite_llm import LiteLlm

gpt_code_agent = Agent(
    name="Joey",
    model=LiteLlm(model="openai/gpt-4o"),
    description=(
        "You answer programming questions."
    ),
    instruction=(
        """You answer questions related to programming.
        Answer coding questions using Python unless told otherwise."""
    )
)
```

Import LiteLLM to use models like GPT, Claude, Grok, etc.

Proprietary + Confidential

Tools provide access to external data and functionality

```
weather_agent = Agent(
    name="Pat",
    model="gemini-#.#-flash",
    description=(
        "Pat the Friendly Weather Agent."
    ),
    instruction=(
        WEATHER_AGENT_INSTRUCTIONS
    ),
    tools=[get_extended_weather_forecast, get_lat_lon]
)
```

Tools are just Python functions

Proprietary + Confidential

Tool functions use type hints and docstring comments as recommended in the Python PEP 8 style guide

```
def get_extended_weather_forecast(lat: float, lon: float) ->
Optional[List[Dict[str, str]]]:
    """
    Fetch the extended weather forecast from the U.S. National Weather Service API
    based on a given latitude and longitude.
    Args:
        lat (float): Latitude of the location (e.g., 38.8977).
        lon (float): Longitude of the location (e.g., -77.0365)
    Returns:
        Optional[List[Dict[str, str]]]: A list of forecast d
        Returns None if data is unavailable or an error occu
    """

                                << CODE OMITTED >>
```

Type hints and docstring comments tell the model how to use the function

Proprietary + Confidential

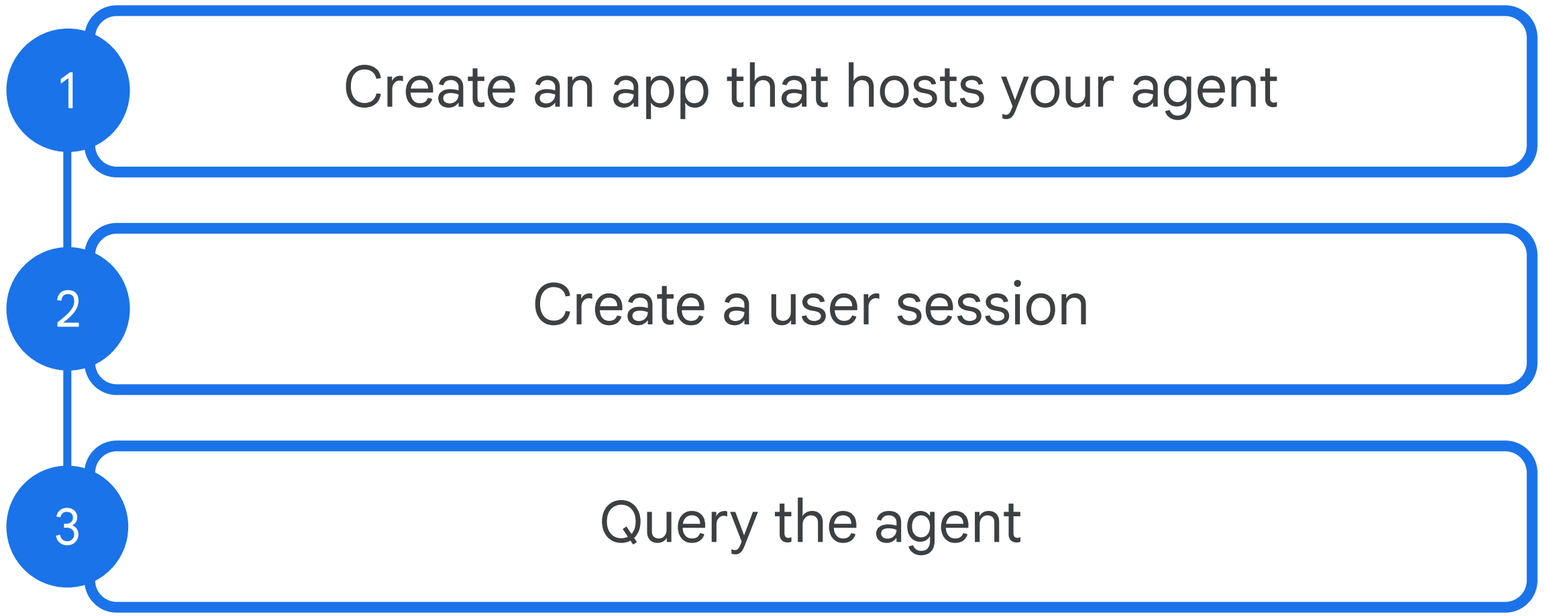
The Google search tool provides up-to-date information

```
from google.adk.tools import google_search

google_search_agent = LlmAgent(
    name="google_search_agent",
    model=MODEL_GEMINI_FLASH,
    description="Provides Google Search Results.",
    instruction=SEARCH_AGENT_INSTRUCTIONS,
    tools=[google_search]
)
```

Proprietary + Confidential

Running your agents



Proprietary + Confidential

1. Create an app that hosts your agent

```
from vertexai.preview import reasoning_engines

app = reasoning_engines.AdkApp(
    agent=code_agent,
    enable_tracing=False,
)
```

Proprietary + Confidential

2. Create a user session

```
user_id = "test-user-id"
session = app.create_session(user_id=user_id)


print(session.id)
```

```
1c4bafc1-31d2-482c-afc6-d357be9868af
```

Proprietary + Confidential

3. Query the agent

```
from IPython.display import Markdown, display
for event in app.stream_query(
    user_id=user_id,
    session_id=session.id,
    message="Write a function to reverse the bits of an integer",
):
    lastevent = event
display(Markdown(lastevent["content"]["parts"][0]["text"]))
```

```
def reverse_bits(n: int, bit_size: int = 32) -> int:
    """Reverses the bits of an unsigned integer.

    Args:
        n: The unsigned integer to reverse.
        bit_size: The number of bits in the integer (default is 32).

    Returns:
        The integer with its bits reversed.
    """
```

Proprietary + Confidential

Model Context Protocol

- Defines how information is structured and shared between agents and models during execution
- Ensures consistent input and output formats for LLM interactions
- Supports structured memory, tool usage, and state tracking across sessions
- Enables agents to collaborate by sharing context and results through a standardized schema
- https://modelcontextprotocol.io/introduction

Proprietary + Confidential

Challenge one: Building an agent with custom tools

Goal: Demonstrate your ability to create and test and agent using the Google Agent Development Kit

Requirements:

- Create an agent that uses tools to retrieve real-time weather data for user locations.
- Provide a weather summary or alert based upon current weather conditions and locations.
- Write test code that demonstrates the agent works for multiple cities in the USA.

Proprietary + Confidential

Challenge one: Instructions

1. Log into the Cloud Skills Boost environment provided by your instructor.
2. Create a Jupyter Notebook using Agent Platform Colab Enterprise.
3. Create a Python function that uses the National Weather Service API to retrieve the weather based on latitude and longitude. Use proper type hints and docstring comments as specified by the Python PEP 8 style guide. (Ask Gemini for help.)
4. Similarly, create a function that uses the Google Maps Geocoding API to convert places to latitude and longitude.
5. Add the functions as tools for and agent using the ADK. Give the agent appropriate instructions.
6. Add support for both Gemini and some other third-party model (Claude, GPT, etc.).
7. Write test code to demonstrate that the agent works for multiple US cities.
8. Upload your notebook to your GitHub repository for grading.

Tips



- See this ADK tutorial: https://google.github.io/adk-docs/tutorials/agent-team/
- Jupyter example: https://colab.research.google.com/github/google/adk-docs/blob/main/examples/python/tutorial/agent_team/adk_tutorial.ipynb
- For more info on tools: https://google.github.io/adk-docs/tools/#tools

Proprietary + Confidential



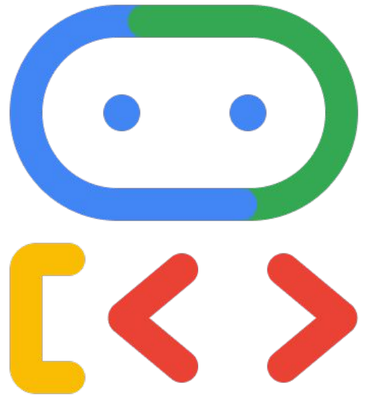
Challenge Two

Enhancing Agents with Callbacks

Proprietary + Confidential

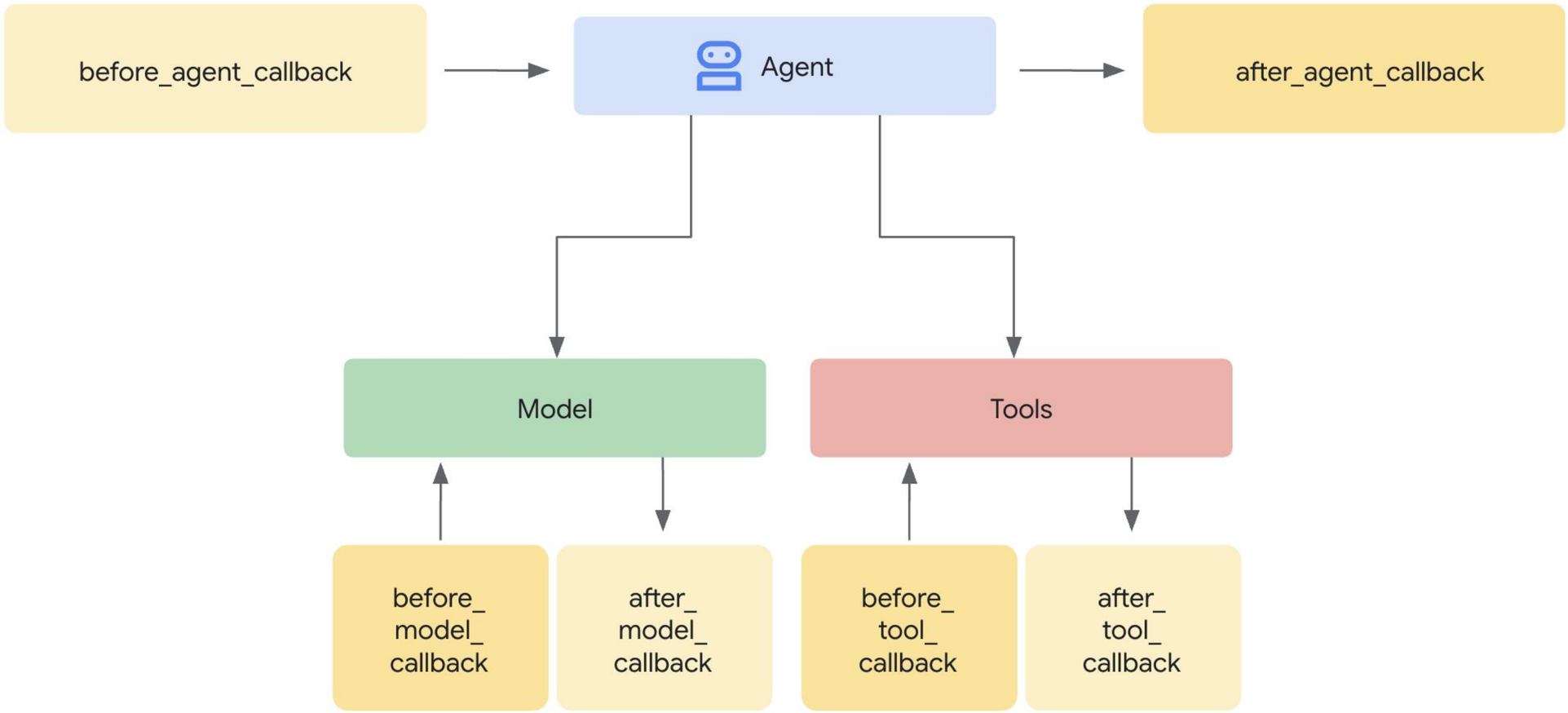
ADK agent lifecycle callbacks

- Agent lifecycle callbacks in ADK allow developers to intercept and customize key stages of an agent’s execution
- Enable pre- and post-processing during agent startup, tool use, and response generation
- Useful for logging, monitoring, and enforcing security policies
- Can be combined with validation steps to ensure safe and reliable agent behavior



Proprietary + Confidential

Callbacks illustrated



Proprietary + Confidential

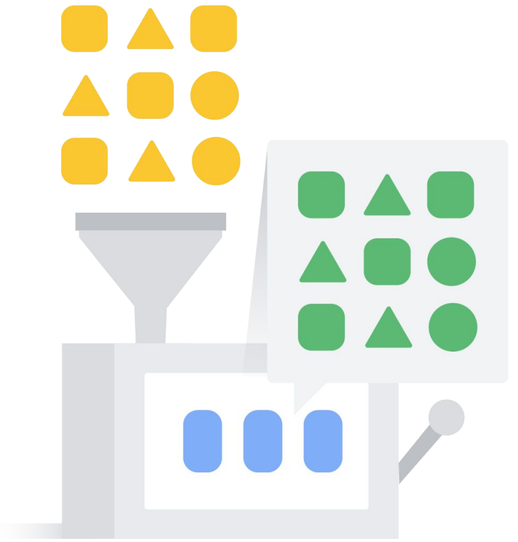
LLM interaction callbacks

- Let you customize how agents interact with the language model at runtime
- Modify prompts before they are sent to the model
  - Add system-level instructions or context dynamically
- Inspect or filter model responses before they are returned to the user
- Useful for debugging, monitoring, and enforcing content policies

Proprietary + Confidential

Prompt validation and filtering

- Prompt validation and filtering help ensure user inputs are safe, relevant, and within defined constraints before reaching the model
- Prevents harmful, off-topic, or malformed prompts from being processed
- Common filters include location checks, keyword restrictions, and input length limits
- Enhances security and reliability of agent interactions



Proprietary + Confidential

Before model callback logging example

The callback signature is important as the system will invoke the function

```
def log_user_prompt(callback_context: CallbackContext, llm_request: LlmRequest)
-> Optional[LlmResponse]:

    if llm_request.contents:
        last = llm_request.contents[-1]
        if last.role == "user" and last.parts and last.parts[0].text:
            logger.info("[%s] USER  » %s", callback_context.agent_name,
            last.parts[0].text.strip())

    return None
```

Return None to allow the processing to continue

Proprietary + Confidential

After model callback logging example

```
def log_model_response(
    callback_context: CallbackContext, llm_response: LlmResponse)
 -> Optional[LlmResponse]:

    if llm_response.content and llm_response.content.parts:
        txt = llm_response.content.parts[0].text
        if txt:
            logger.info("[%s] MODEL » %s", callback_context.agent_name, txt.strip())

    return None
```

Proprietary + Confidential

Validating user input (prompts)

```
def moderate_user_prompt(callback_context: CallbackContext,llm_request: LlmRequest
) -> Optional[LlmResponse]:
    try:
        user_text = last.parts[0].text.strip()
        result_text = check_user_input(user_text)

        if result_text.strip().upper() == "BAD":
            return LlmResponse(content={
                "role": "model",
                "parts": [{"text": "Message violates our content guidelines."}]})
    except Exception as e:
        import logging
        logging.exception("Moderation callback failed: %s

    return None
```

Returning a response stops further processing

Proprietary + Confidential

Chaining multiple callbacks together allows more than one function for the same event

```
def chained_before_callback(callback_context, llm_request):
    # 1. Moderation check
    moderation_result = moderate_user_prompt(callback_context, llm_request)
    if moderation_result is not None:
        return moderation_result  # STOP: message was inappropriate

    # 2. Log user input (optional)
    log_user_prompt(callback_context, llm_request)

    return None  # Allow agent to proceed
```

Proprietary + Confidential

Configuring agents to use callbacks

```
weather_agent_with_moderation = LlmAgent(
    name="Pat",
    model="gemini-2.0-flash",
    description="Pat the Friendly Weather Agent.",
    instruction=(WEATHER_AGENT_INSTRUCTIONS),
    tools=[get_extended_weather_forecast, get_lat_lon],
    before_model_callback=chained_before_callback,
     after_model_callback=log_model_response,
)
```

Proprietary + Confidential

Challenge two: Enhancing agents with callbacks

Goal: Demonstrate your ability to use callback functions to add logging and validation to your ADK agents.

Requirements:

- Use a callback function to log user prompts.
- Use a callback function to log model responses.
- Use a callback function to check and validate user input before sending requests to then model.

Proprietary + Confidential

Challenge two: Instructions

1. Make a copy of the Jupyter notebook you created in the previous challenge.
2. Add callback functions to log both user prompts and model responses.
3. Add a callback function to validate user input.
    a. Ensure the user's location is in the United States (the NWS API does not support non-US locations).
    b. Ensure the user input is not malicious prior to sending the request to the model.
4. Upload your notebook to your GitHub repository for grading.

Tips



- To avoid hard to debug errors:
  - Start with a working agent without callbacks
  - Add a callback function that just prints a message (This makes sure the signature is correct)
  - Refactor complex logic in separate, testable functions
  - Do one callback at a time
  - Keep testing at every step
- See the docs: https://google.github.io/adk-docs/callbacks/

Proprietary + Confidential



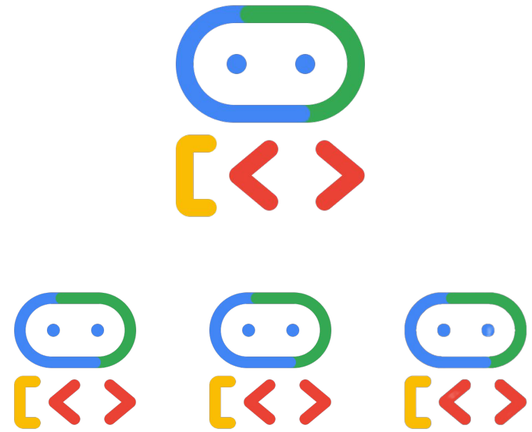
Challenge Three

Developing Multi-Agent Systems

Proprietary + Confidential

Multi-agent systems

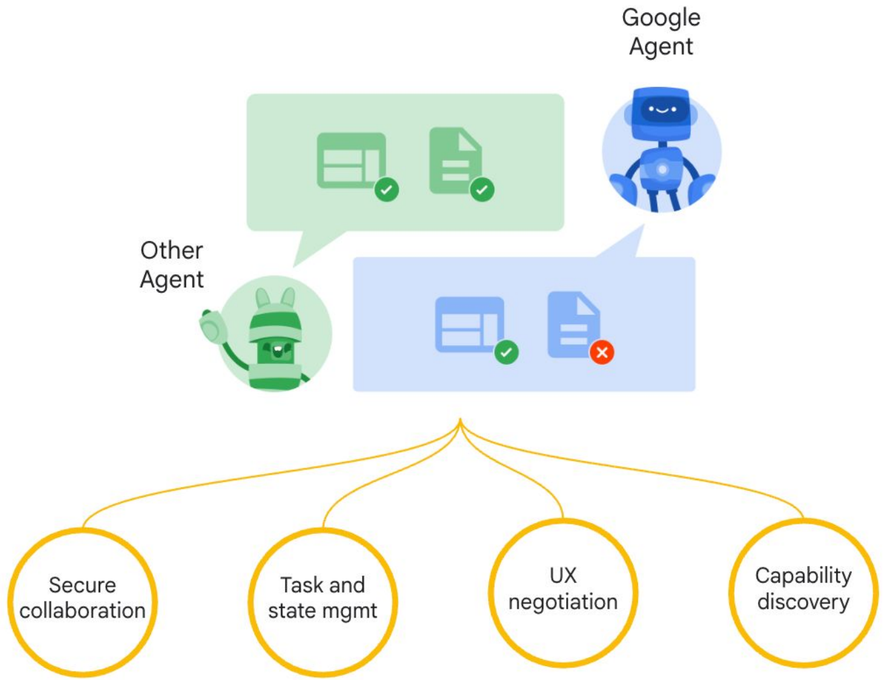
- Multi-agent systems coordinate multiple specialized agents to handle complex tasks more effectively
- Each agent focuses on a specific function, such as data retrieval, analysis, or validation
- A coordinating agent manages communication and task delegation between sub-agents
- Enables scalable and modular solutions by dividing responsibilities across agents



Proprietary + Confidential

Agent-to-Agent (A2A) Protocol

- The Agent-to-Agent (A2A) Protocol enables structured communication and coordination between agents in a multi-agent system
- Defines how agents share messages, results, and task context with one another
- Supports chaining agents in sequences or parallel flows to complete workflows
- Ensures consistency and traceability across inter-agent interactions
- https://google.github.io/A2A/



Proprietary + Confidential

Sub agents

- Sub agents are specialized agents that handle focused tasks within a multi-agent system
- Operate under the direction of a main coordinating agent
  - Allow decomposition of complex workflows into manageable parts
  - Improve modularity, reusability, and clarity in agent-based applications
- Sub-agents are programmed just like any other agents
- Can mix different model types and model versions
  - Use smaller models for simpler tasks
  - Use other models (GPT, Claude, tuned models) for specialized tasks
- Enabled by the A2A protocol

Proprietary + Confidential

Configuring models to use sub agents

```
main_agent = LlmAgent(
    name="main_agent",
    model=MODEL_GEMINI_FLASH,
    description="Provides Answers to Users Questions.",
    instruction=MAIN_AGENT_INSTRUCTIONS,
    tools=[agent_tool.AgentTool(agent=google_search_agent)],
    sub_agents=[weather_agent],
)
```

You can create sub-agents and/or use agents as tools

Proprietary + Confidential

Challenge three: Developing multi-agent systems

Goal: Demonstrate your ability to program multi-agent systems using the Google ADK

Requirements:

- Build a multi-agent system with three agents:
  - The coordinating or root agent
  - A weather agent
  - A search agent
- The root agent receives user requests and delegates tasks appropriately to its sub agents

Proprietary + Confidential

Challenge three: Instructions

1. Make a copy of the Jupyter notebook you created in the previous challenge to use as a starting point.
2. Add a search agent that uses the ADK built-in Google Search tool.
3. Add a root agent that uses the weather and search agents as sub agents.
4. Write code to test your agents. Output events to your notebook that demonstrate the use of the sub agents.
5. Upload your notebook to your GitHub repository for grading.

Proprietary + Confidential

Tips



- See the docs: https://google.github.io/adk-docs/agents/multi-agents/#11-agent-hierarchy-parent-agent-sub-agents

Proprietary + Confidential

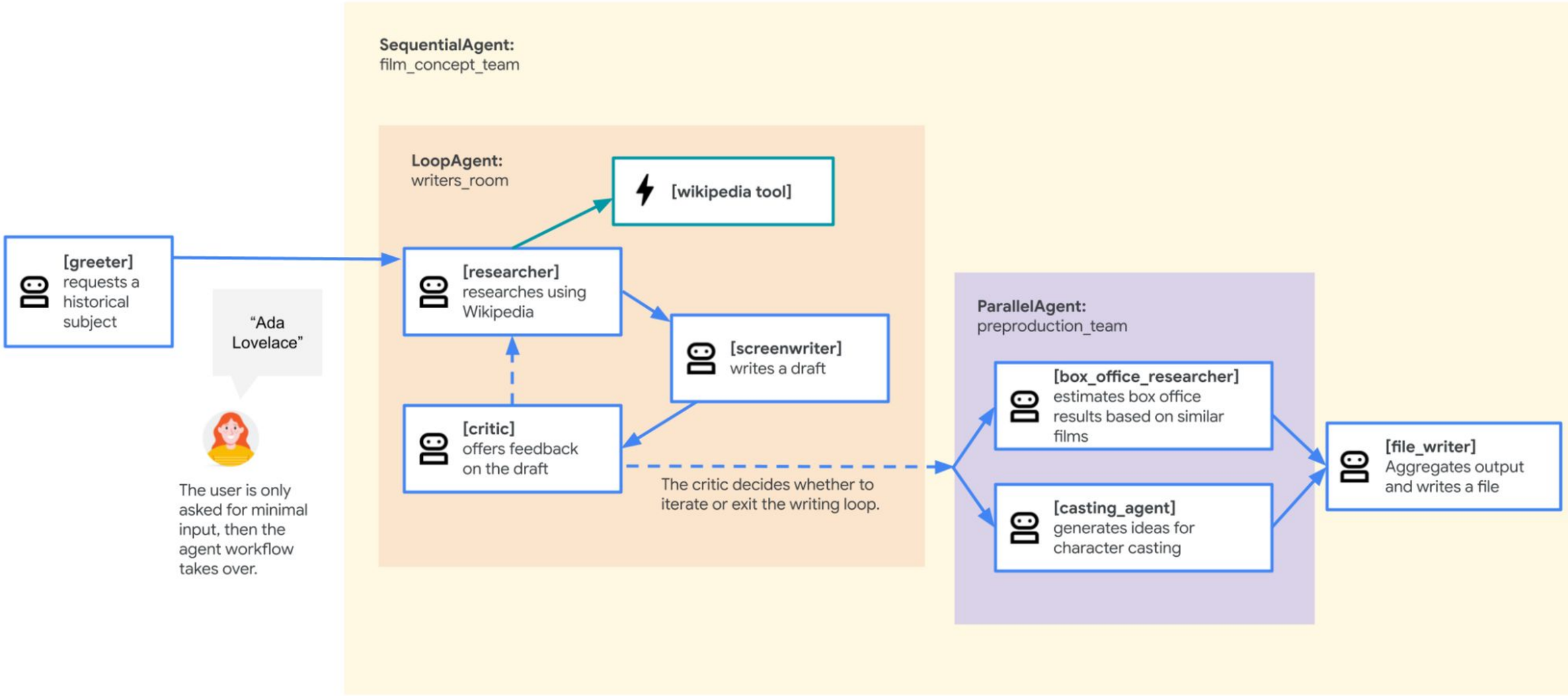


Challenge Four

Programming an Agent Workflow

Proprietary + Confidential

Workflow agents example



Proprietary + Confidential

Sequential agents

- Execute tasks in a defined order, passing outputs from one as inputs to the next
- Each agent completes its task before the next begins
- Ensures logical flow and dependency handling across multiple tasks

```
film_concept_team = SequentialAgent(
    name="film_concept_team",
    description="Write a film plot...",
    sub_agents=[
        researcher,
        screenwriter,
        file_writer
    ],
)
```

Proprietary + Confidential

Loop agents

- Repeat a task or sequence of tasks until a specific condition is met
- Useful for iterative processes like data refinement, retries, or continuous monitoring
- Can be configured with custom exit conditions based on agent output or user input

```
writers_room = LoopAgent(
    name="writers_room",
    description="Iterates ...",
    sub_agents=[
        researcher,
        screenwriter,
        critic
    ],
    max_iterations=3,
)
```

Proprietary + Confidential

Parallel agents

- Run multiple agents at the same time to speed up processing of independent tasks
- Improves efficiency by reducing wait time in multi-agent workflows
- A coordinating agent can merge or evaluate results once all tasks complete

```
preproduction_team = ParallelAgent(
    name="preproduction_team",
    sub_agents=[
        box_office_researcher,
        casting_agent
    ]
)
```

Proprietary + Confidential

Greeter (Root) agents

- Primary entry point for user interactions in an agentic system
- Receives the initial message and determines how to handle or route the request
- May respond directly or delegate tasks to sub-agents
- Maintains session context and orchestrates the overall user experience

```
root_agent = Agent(
    name="greeter",
    model=model_name,
    description="Craft a movie plot.",
    instruction="""
    ...
    """,
    tools=[append_to_state],
    sub_agents=[film_concept_team],
)
```

Proprietary + Confidential

Maintaining state

- Allows agents to remember context across interactions and coordinate long-running workflows
- Stores information like user input, past messages, or task progress
- State can be persisted in memory, databases, or session services depending on use case

```
def append_to_state(tool_context, field, response):
    existing_state = tool_context.state.get(field, [])
    tool_context.state[field] = existing_state + [response]
    return {"status": "success"}
```

Proprietary + Confidential

Challenge four: Programming an agent workflow

Goal: Demonstrate your ability to program a complex process using ADK workflow agents

Requirements:

- Create an agent that answers questions but verifies and refines the answers prior to returning them.
- Use a workflow with multiple agents to answer, verify, and refine responses.

Proprietary + Confidential

Challenge four: Instructions

1. Copy of one of your previous Jupyter notebooks to use as a starting point.
2. Add Greeter, Search, Critique, and Refine agents.
3. Use either a Sequential or Loop agent to build the answer team.
    a. The Search agent finds the data to answer the question
    b. The critique agent looks at the initial response and makes suggestions as to how it can be improved
    c. The Refine agent rewrites the response based on the improvements suggested
4. Write code to test your agents. Output events to your notebook that demonstrate the use of the sub agents.
5. Upload your notebook to your GitHub repository for grading.

Proprietary + Confidential

Tips



- See the docs:
- https://google.github.io/adk-docs/agents/workflow-agents/

Proprietary + Confidential



Bonus Challenge Five

Deploying Agents

Proprietary + Confidential

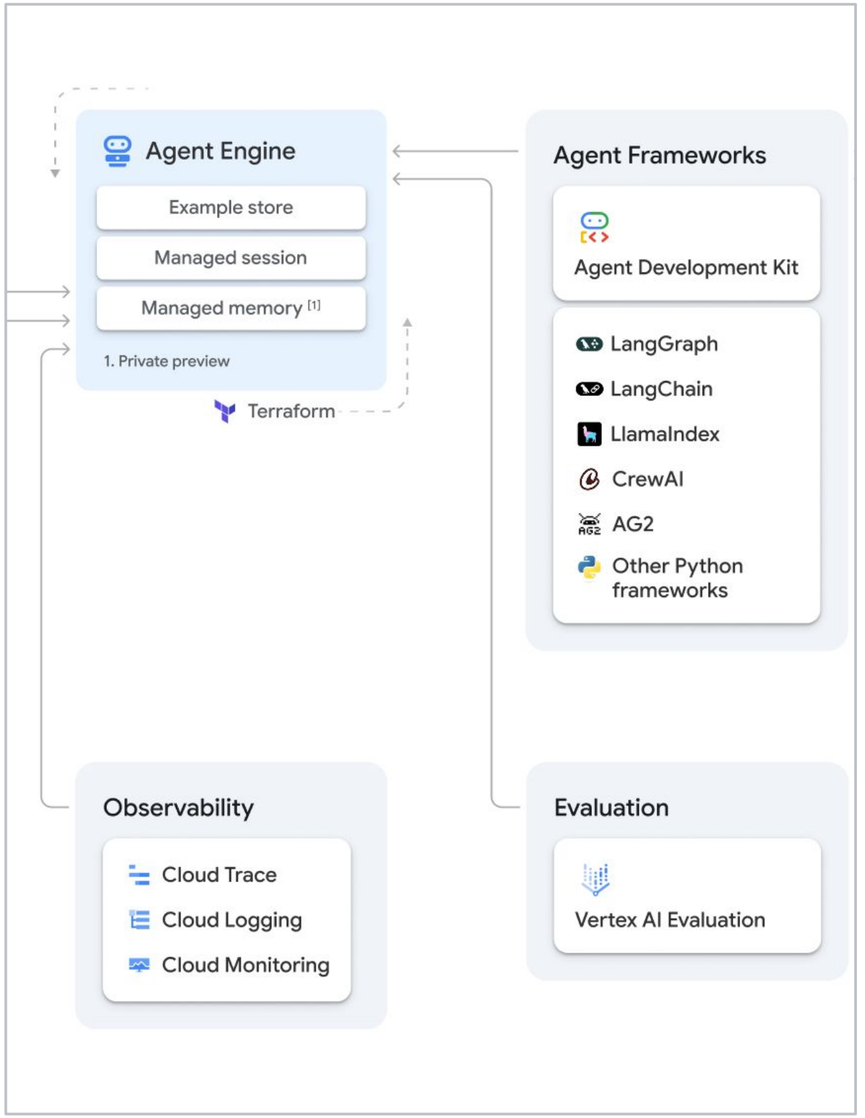
Deploying ADK agents

- Deploying ADK agents makes them accessible in real-world applications and scalable environments
- Agents can be deployed using Google Agent Platform or custom runners
- Supports integration with Cloud Run for serving agents as web services
- Enables web apps, APIs, and chat interfaces to interact with deployed agents

Proprietary + Confidential

Agent Platform Deployments

- Agent Platform Deployment is a managed platform for running and scaling ADK agents on Google Cloud
- Provides infrastructure to deploy, invoke, and monitor agents at scale
- Supports session management, tracing, and multi-agent coordination
- Integrates with Agent Platform, and other Google Cloud services for production-grade solutions
- Supports third-party frameworks like LangChain, LangGraph, and others



Proprietary + Confidential

Steps to deploy an agent to Agent Platform

1. Initialize Agent Platform with project and location settings
2. Define your agent using the ADK
3. Test your agent in your local environment
4. Use `agent_engines.create()` to deploy the agent and manage sessions and requests

Proprietary + Confidential

1. Initialize Vertex AI

```
import vertexai

vertexai.init(
    project=PROJECT_ID,
    location=LOCATION,
    staging_bucket=STAGING_BUCKET,
)
```

Proprietary + Confidential

2. Define your agent

```
search_agent = Agent(
    name="Google",
    model="gemini-#.#-flash",
    description=(
        "You can search Google..."
    ),
    instruction=(
        """You are a helpful Chatbot."""
    ),
    tools=[google_search]
)
```

Proprietary + Confidential

3. Test your agent locally

```
from vertexai.preview.reasoning_engines import AdkApp

app = AdkApp(agent=search_agent)

for event in app.stream_query(
    user_id="USER_ID",
    message="Give me the news highlights in the world
of sports.",
):
    print(event)
```

Proprietary + Confidential

4. Deploy your agent to Agent Platform

```
from vertexai import agent_engines

remote_agent = agent_engines.create(
    app,
    requirements=["google-cloud-aiplatform[agent_engines,adk]"],
)
```

```
for event in remote_agent.stream_query(
    user_id="agent-engine-test-user",
    message="Give me the news highlights in the world of sports.",
):
    print(event)
```

```
{'content': {'parts': [{'text': "Here are some of the latest sports n
```

Proprietary + Confidential

Challenge five: Deploying an agent to Agent Platform

Goal: Demonstrate your ability to deploy and use an agent using Google Agent Platform

Requirements:

- Create an agent using the ADK.
- Deploy the agent to Agent Platform
- Test the agent.

Proprietary + Confidential

Challenge five: Instructions

1. Make a copy of the Jupyter notebook you created in the previous challenge to use as a starting point. You will deploy that agent to Agent Platform.
2. Write the code to deploy the agent to Agent Platform.
3. Write test code to ensure it works.
4. Upload your notebook to your GitHub repository for grading.

Tips



- See the docs: https://docs.cloud.google.com/gemini-enterprise-agent-platform/scale
- Tutorial: https://docs.cloud.google.com/gemini-enterprise-agent-platform/scale/runtime/deploy-an-agent

Proprietary + Confidential

06

Case Study

Federal Emergency Machine Assistant

FEMA

Proprietary + Confidential

Project ReadyNow!

You've been hired by the Federal Emergency Management Agency (FEMA) to build an Emergency Preparedness Chat Agent. The project is code-named, "ReadyNow!".

The goal is to help people get real-time updates during a disaster so they know what's going on, where to go, and how to stay safe. The agent must use weather data, search the internet, suggest evacuation routes, and provide safety information based on the user's location and current situation.



Proprietary + Confidential

Key stakeholder requirements

FEMA has asked that you provide an initial proof of concept that includes the following key capabilities.

- Real-time weather and news alerts
- In the event of a disaster, provide suggested routes to safety
- Log all interactions between the user and the agent
- Validate that user input is appropriate and refuse requests not related to the agent’s mission
- Ensure agent responses are valid, well-written, and easy to understand



Proprietary + Confidential

Case Study: Federal Emergency Machine Assistant

Goal: Demonstrate your ability to build a complex multi-agent system using the Google ADK

Requirements:

- A root agent that describes what the agent can do and coordinates tasks and sub-agents
- Agents for weather forecasting, searching the internet, providing routes using the Google Maps API, and answering questions
- A sequential workflow that validates and refines responses from the agent
- Callbacks functions to log all user-agent interactions
- User input validation
- Deploy the agent to Agent Platform
- Test code that demonstrate the agent’s functionality

Proprietary + Confidential

Case Study: Instructions

1. Create an architecture diagram that depicts your solution to the ReadyNow! case study.
2. Implement the case study requirements using a Colab Enterprise Jupyter Notebook.
3. Program your solution any way you like. Use the code you created from the earlier challenges as a guide. Also, feel free to ask Gemini for help.
4. Test your agent locally.
5. Deploy your agent to Agent Platform.
6. Write test code to demonstrate a working, deployed solution.
7. Upload all artifacts to your GitHub repository for grading.

Thank you.